NJOY EMLAK

Real Estate Portfolio Management System

TECHNICAL REPORT

Project	Njoy Emlak — Real Estate Portfolio Management System
Database Engine	SQLite
Data Source	njoy.sahibinden.com (Live Active Portfolio)
Date	April 2025

TEAM MEMBERS

Altay Yeles	**Alper Güner**	**Mustafa Ulaş Karaaslan**	**Mehmet Can Öztürk**	**Emre Aslan**

1. Introduction and Project Idea

This project was initiated to design a relational database for the actively operating Istanbul-based real estate company Njoy Emlak (njoy.sahibinden.com), owned by team member Alper Güner's uncle. The system consolidates listing, agent, and property data into a single normalized database and enables business intelligence reporting from that data.

All listing data in the database was manually collected from njoyemlak.com through data scraping. Five active listings, along with all of their interior features, exterior features, and orientation details, were entered into the system. Because the data comes from a live source, no artificial or fabricated content was used.

2. Problem Statement and Importance

2.1 Problem Statement

Small and mid-sized real estate offices struggle to manage listing, agent, and feature data in an organized way as their portfolio grows. Dependency on scattered Excel files or ready-made third-party platforms leads to critical operational issues, including data redundancy, update anomalies, and insufficient reporting capabilities.

2.2 Why This Project Matters

Implementing this relational database system delivers the following concrete benefits:

- Centralized Data Management: All listing, agent, and feature data is stored in one place — updates are made at a single point.
- Powerful Filtering: Instant queries by budget, area, district, and property features (elevator, AC, orientation, etc.) become possible.
- Performance Analysis: The number of listings per agent and the total portfolio value can be calculated instantly.
- Scalability: New listing types, feature categories, or staff members can be added to the system with ease.
- Platform Independence: SQLite's file-based structure works in any environment and integrates with any application.

3. Database Schema Design

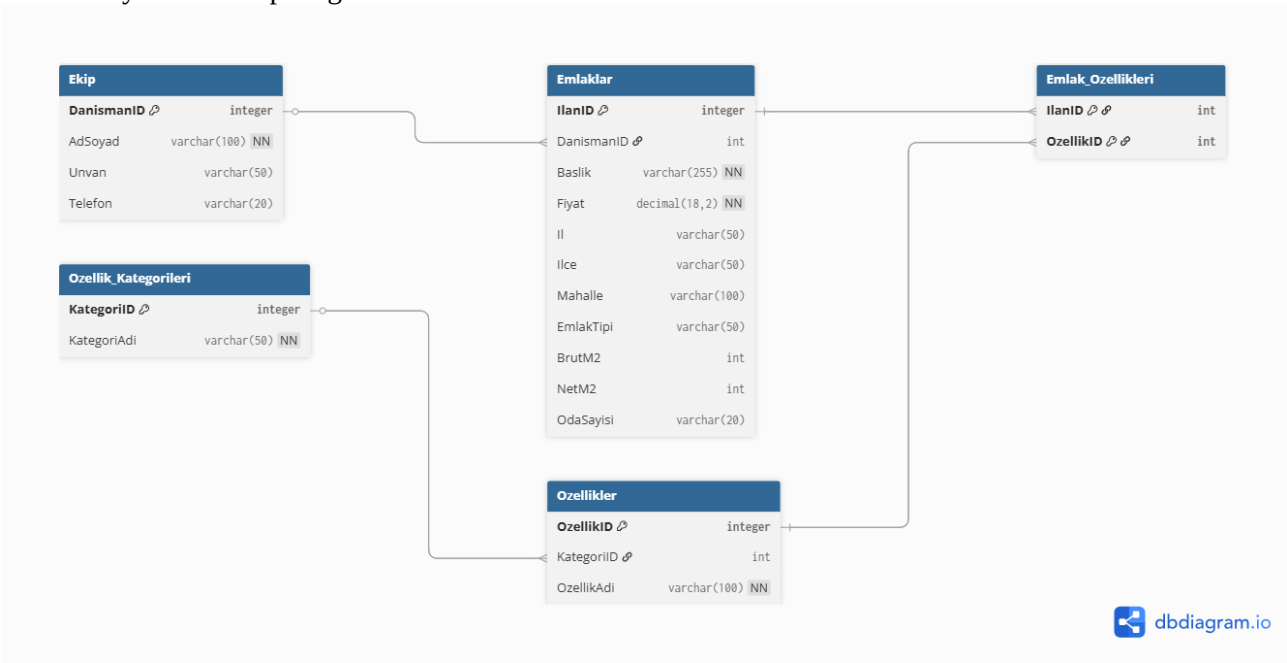
3.1 Tables and Relationships

Five tables have been designed targeting Third Normal Form (3NF). Relationships between tables are defined through Foreign Keys. The many-to-many (M:N) relationship between Listings and Features is resolved via a separate junction table.

Table	Key Columns	Description
Ekip (Team)	DanismanID (PK, AUTO) AdSoyad, Unvan, Telefon	Stores real estate agents and the store owner working at the office.
Emlaklar (Listings)	IlanID (PK) DanismanID (FK) Fiyat, Il, Ilce, EmlakTipi BrutM2, NetM2, OdaSayisi	Holds the core details of all rental and sale property listings in the portfolio.
Ozellik_Kategoriler i (Feature Categories)	KategoriID (PK, AUTO) KategoriAdi	Groups property features into categories: Interior, Exterior, Orientation.
Ozellikler (Features)	OzellikID (PK, AUTO) KategoriID (FK) OzellikAdi	A pool of all 33 features such as Elevator, Air Conditioning, Steel Door, South-Facing.
Emlak_Ozellikleri (Listing-Features)	IlanID (FK — Emlaklar) OzellikID (FK — Ozellikler) [Composite PK]	Junction table resolving the many-to-many (M:N) relationship between listings and features.

3.2 Relationship Diagram

The entity-relationship diagram below illustrates the normalized structure of the schema:



The detailed version of the ERD can be found in SCHEMA.md in the project repository.

4. Planned SQL Features

The following SQL features are planned for implementation in upcoming phases of the project. Features marked [Planned] are architectural targets not yet added to the current schema.

SQL Feature	Purpose
JOIN (INNER, LEFT, Multi-table)	Joining tables; matching agents with their listings
WHERE + IN / BETWEEN	Multi-criteria filters by district, price range, and listing type
GROUP BY + HAVING	Per-agent portfolio analysis with filtered grouping
COUNT, SUM, AVG	Listing count, total portfolio value, average price per m²
ORDER BY	Sorting results by price, area, or date
CTE [Planned]	Making complex queries modular, readable, and manageable with temporary result sets
VIEW [Planned]	Turning frequently used complex queries into reusable virtual tables
INDEX [Planned]	Adding indexes on price and district columns for faster queries

5. Sample SQL Queries

Query 1 — Full Listing Report (INNER JOIN)

Lists all active listings with the responsible agent's name and contact information. This is the primary output query for clients or office managers.

```
SELECT E.Baslik, E.Fiyat, E.Ilce, E.EmlakTipi,
       K.AdSoyad AS Danisman_Adi, K.Telefon
FROM   Emlaklar E
INNER JOIN Ekip K ON E.DanismanID = K.DanismanID;
```

Query 2 — Budget and Region Filtering (WHERE, IN)

Retrieves apartments with a budget of 40,000 TL or less in the Şişli and Beyoğlu districts (İstiklal, Cihangir, Firuzağa, etc.).

```
SELECT Baslik, Fiyat, Ilce, Mahalle
FROM   Emlaklar
WHERE  Fiyat <= 40000
  AND  Ilce IN ('Şişli', 'Beyoğlu')
ORDER BY Fiyat DESC;
```

Query 3 — Agent Portfolio Analysis (GROUP BY + SUM + COUNT)

A performance report designed for management. The LEFT JOIN ensures agents with no listings are also included. Each agent's listing count and total portfolio value (TRY) is calculated automatically.

```
SELECT K.AdSoyad,
       COUNT(E.IlanID)  AS ToplamIlanSayisi,
       SUM(E.Fiyat)      AS ToplamPortfoyDegeri
FROM   Ekip K
LEFT JOIN Emlaklar E ON K.DanismanID = E.DanismanID
GROUP BY K.AdSoyad;
```

Query 4 — Feature-Based Filtering (4-Table JOIN + DISTINCT)

Lists properties with heat insulation or air conditioning along with their feature category. This query chains four tables through JOIN operations, demonstrating the practical value of the junction table design.

```
SELECT DISTINCT E.Baslik, E.Fiyat,
                Oz.OzellikAdi, Kat.KategoriAdi
FROM   Emlaklar E
INNER JOIN Emlak_Ozellikleri EO       ON E.IlanID     = EO.IlanID
INNER JOIN Ozellikler Oz              ON EO.OzellikID = Oz.OzellikID
INNER JOIN Ozellik_Kategorileri Kat  ON Oz.KategoriID = Kat.KategoriID
WHERE Oz.OzellikAdi IN ('Isı Yalıtımı', 'Klima');
```

Query 5 — Rent Cost per m² (Arithmetic)

Calculates the monthly rent cost per gross and net square meter for each listing, enabling quick price-per-m² comparison across the portfolio.

```
SELECT Baslik, Fiyat, BrutM2, NetM2,
       ROUND(Fiyat / BrutM2, 0) AS BrutM2_Fiyati,
       ROUND(Fiyat / NetM2, 0)  AS NetM2_Fiyati
FROM   Emlaklar
WHERE  BrutM2 > 0 AND NetM2 > 0
ORDER BY NetM2_Fiyati DESC;
```

6. Conclusion

In this Milestone phase, the normalized database schema — built with real-world data from the active Njoy Emlak portfolio — has been successfully completed. Table relationships, junction table design, and all core SQL queries have been implemented and validated.

In upcoming phases, advanced SQL features such as CTEs, VIEWs, and INDEXes will be integrated into the system, and the dataset will grow alongside the actual company portfolio. A Flask-based web application with an Art Deco UI and REST API endpoints has also been developed as part of the extended scope, enabling browser-based interaction with the database.

GitHub Repository	**Live Data Source**
github.com/alpergnr/SQL-PROJECT	njoy.sahibinden.com